Two Pointers

⇒ [Good Segment Technique]

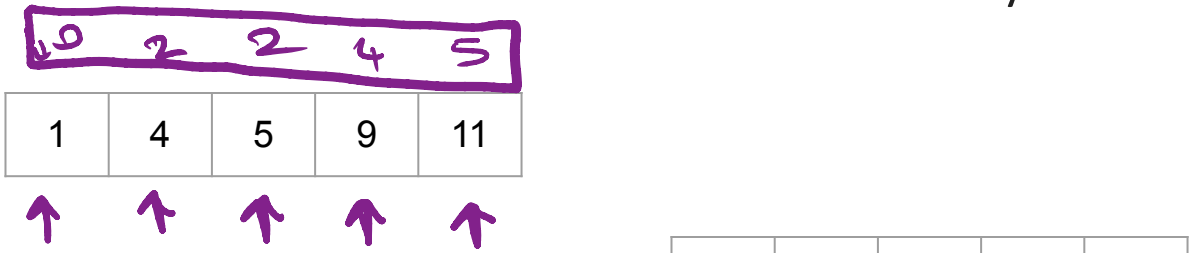
- Gaurish Baliga

Two Pointers

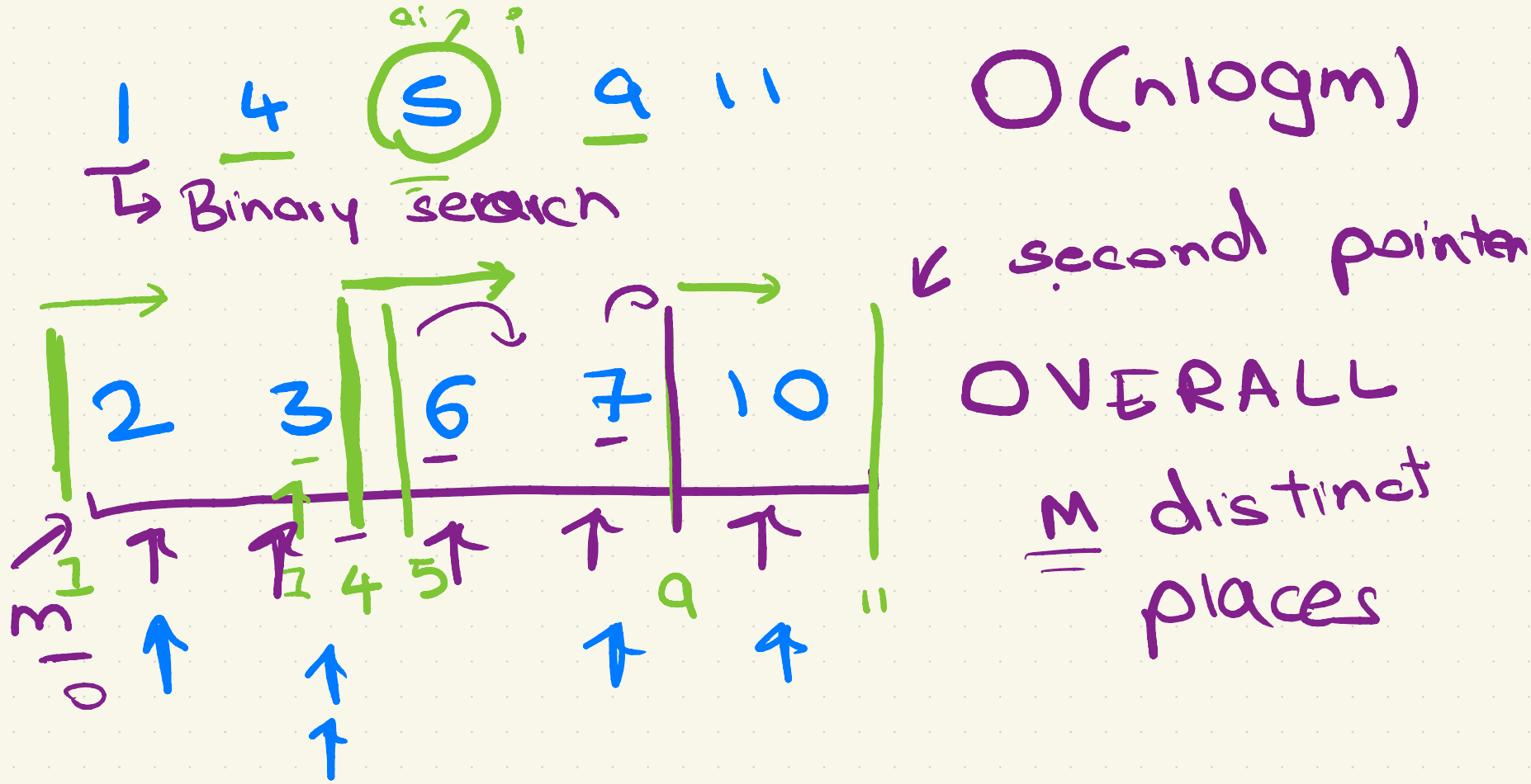
- Widely used in Competitive Programming
- Optimization Technique
- Most Two Pointer problems can be solved using Binary Search
- Useful for a lot of array based problems
- Useful for interviews too

Problem

- Given 2 sorted arrays of size N and M, for each element in 1st array find number of elements smaller than that in the 2nd array

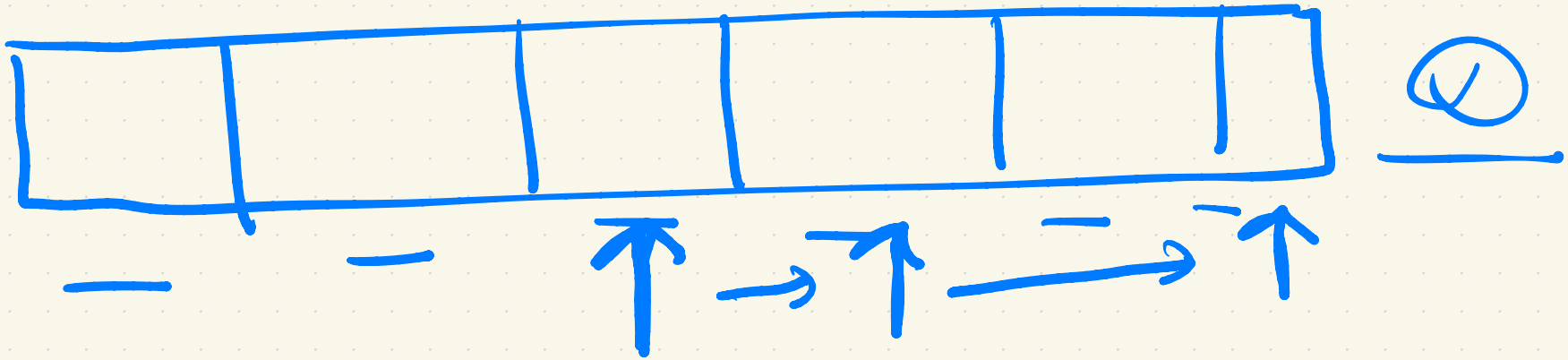


O(NlogM) with Binary Search?


a[2]
i
1 4 5 9 11
Binary search
O(nlogm)
second pointer
2 3 6 7 10
OVERALL
M distinct
places
m
1
0
1
4
5
9
11

over all p

10,000



Solution using 2 Pointers:

If 5 elements are smaller than a[i], how many elements will be lesser than a[i + 1]?

Clearly, we should check for elements bigger than first 5 elements now as a[i + 1] >= a[i]

Having 2 pointers and both only move right. Time complexity?

```
vector<int> a(n), b(m);
vector<int> ans(n);
int i = 0, j = 0;
while(i < n){
    while(j < m && b[j] < a[i]){
        j++;
    }
    ans[i] = j;
    i++;
}
```

O(n+m)

O(m)

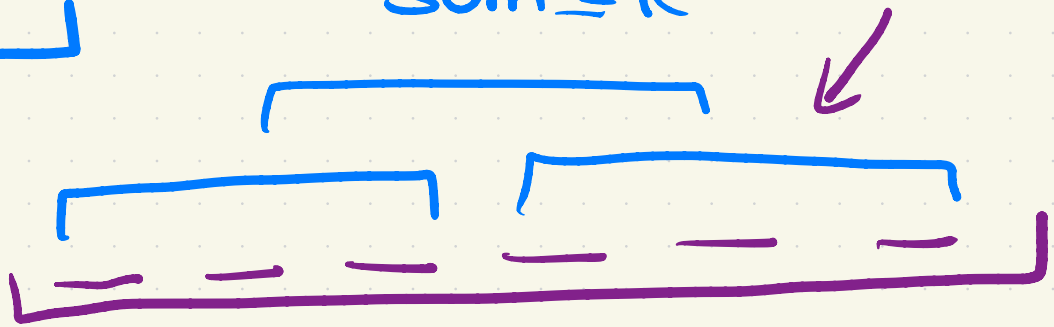
Good Segments Technique Type 1

- Let’s define a **Good Segment** as a subarray that follows a particular **property**.
- Now ask yourself, can you prove that all segments enclosed with a good segment are also good, which means all subarrays enclosed in a good subarray also follow the **property**.
- Ex: If an array only contains positive integers and we define a good segment as a subarray with a sum <= K, then all subarray enclosed within a good subarray automatically become good.

Type 1:

A segment is called good if the sum of its elements is $\leq k$

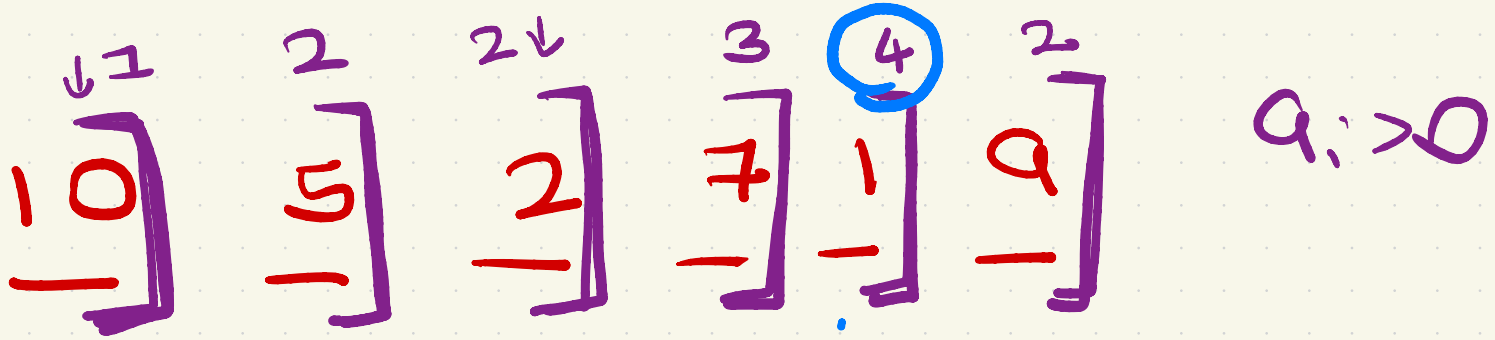
$a_i \geq 1$



Problem

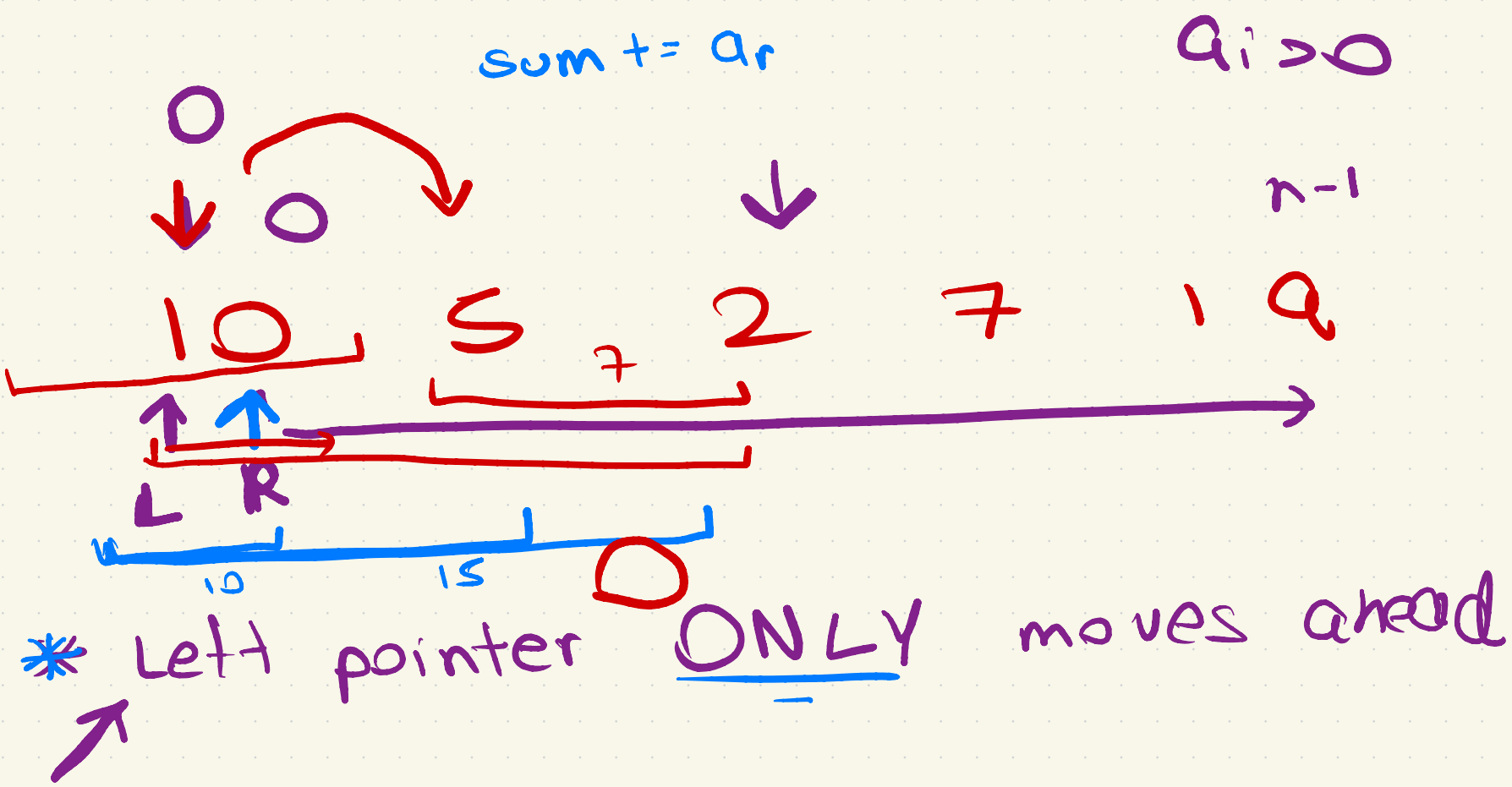
Given an array of N positive elements, find out the length of the longest subarray with sum <= K.

Input: *arr[] = { 10, 5, 2, 7, 1, 9 }, k = 15*
Output: *4*
Explanation: *The sub-array is* ***{5, 2, 7, 1}****.*



q; >0

Binary search + prefix sum O(nlogn)

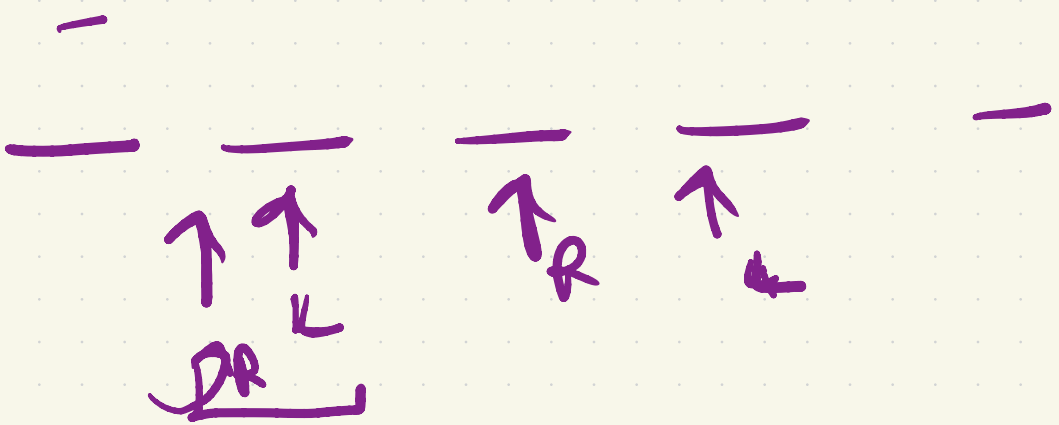


$1 - 100 \rightarrow [100 - 1 + 1]$

$S_2\ [l, r] \rightarrow r - l + 1$

$k = 4$

Is this a problem?!



$(R - L + 1)$

$(R - (R+1) + 1)$

$(-1 + 1)$

$[0]$

Approach 1 - Binary Search

For each index **[i]** (0 <= i < n) find out the farthest index **[j]** to the right upto which the sum remains less than K.

Answer = highest value of j - i + 1 across all values of i

Can we do this using Binary Search + Prefix sums?
Time complexity: O(NlogN)

Solution using 2 Pointers

TC → O(n)

If the current segment is not good any segment bigger will never be good, so remove some elements to make it good.

If current segment is good, try adding a new element and check if it still remains good.

Remove elements from left, add elements from right.

```cpp
int n, k;
cin >> n >> k;
vector<int> a(n);
for(int i = 0; i < n; i++)
    cin >> a[i];
int ans = 0, sum = 0;
int left = 0, right = 0;
while(right < n){
    sum += a[right]; // include a[right]
    while(left <= right && sum > k){
        // remove a[left]
        sum -= a[left];
        left++;
    }

    // if current segment is good, try updating ans
    if(sum <= k)
        ans = max(ans, right - left + 1);
    right++; // move right pointer 1 step right
}
cout << ans << endl;
```

Problem

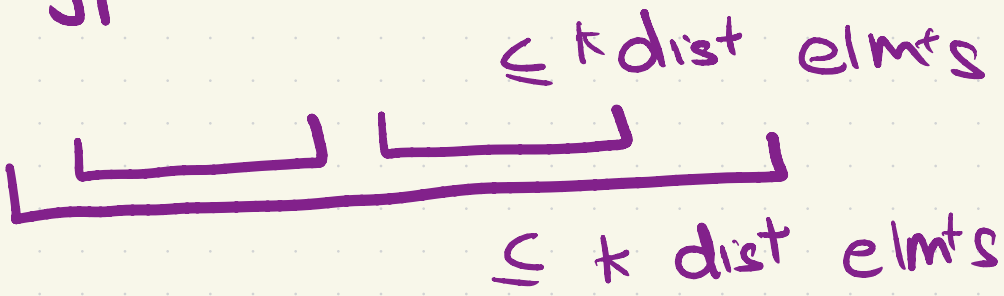
code this out + think of solution 2.5pm

Given an array of N elements, find out the length of the longest subarray with number of distinct elements <= K

??

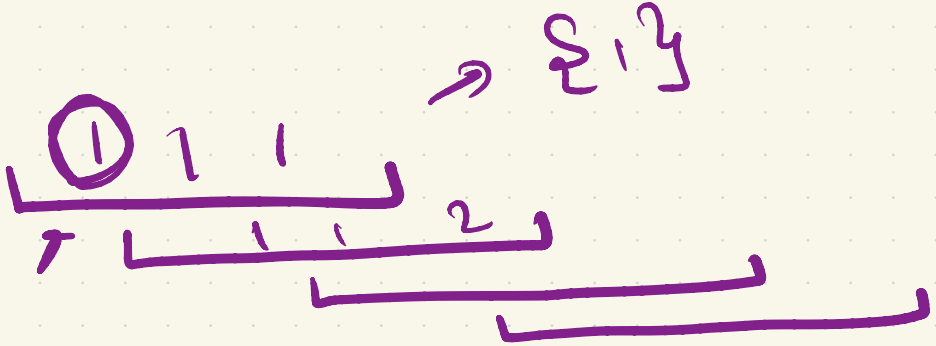
Input: *arr[] = { 10, 5, 2, 7, 2, 2, 1, 9 }, k = 3*
Output: *5*
Explanation: *The sub-array is* ***{2, 7, 2, 2, 1}***

→ Type 1



→ Left point will always move → right

→ Maintain the number of dist. elements in a range



Set ✗

→ number of keys = number of distinct elements

Map ✓

Solution using 2 Pointers

TC → O(n log k)

If the current segment is not good any segment bigger will never be good, so remove some elements to make it good.

If current segment is good, try adding a new element and check if it still remains good.

Remove elements from left, add elements from right.

```cpp
int n, k;
cin >> n >> k;
vector<int> a(n);
for(int i = 0; i < n; i++)
    cin >> a[i];
int ans = 0;
map<int, int> freq;
int left = 0, right = 0;
while(right < n){
    freq[a[right]]++; // include a[right]
    while(left <= right && freq.size() > k){
        // remove a[left]
        freq[a[left]]--;
        if(freq[a[left]] == 0)
            freq.erase(a[left]);
        left++;
    }

    // if current segment is good, try updating ans
    if(freq.size() <= k)
        ans = max(ans, right - left + 1);
    right++; // move right pointer 1 step right
}
cout << ans << endl;
```

map

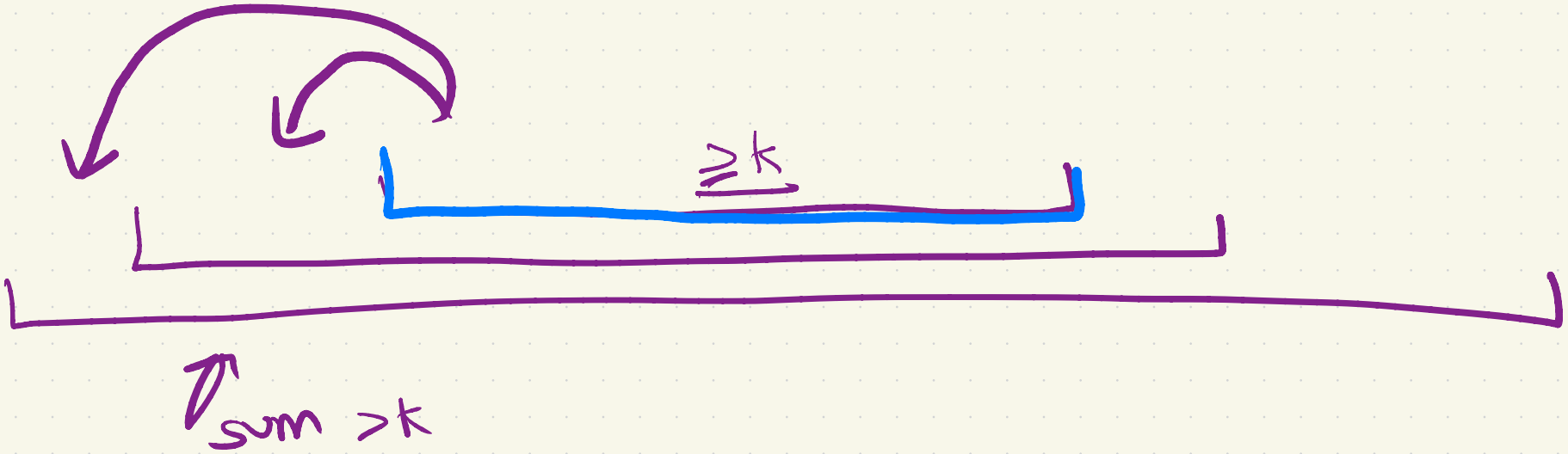
unique elms

Good Segments Technique Type 2

- Let's define a **Good Segment** as a subarray that follows a particular **property**.
- Now ask yourself, can you prove that all segments enclosing a good segment are also good, which means all subarrays enclosing a good subarray also follow the **property**.
- Ex: If an array only contains positive integers and we define a good segment as a subarray with a sum >= K, then all subarray enclosing a good subarray automatically become good.

Type 2 : *usually its shortest

→ subarrays with sum ≥ k , $a_i > 0$



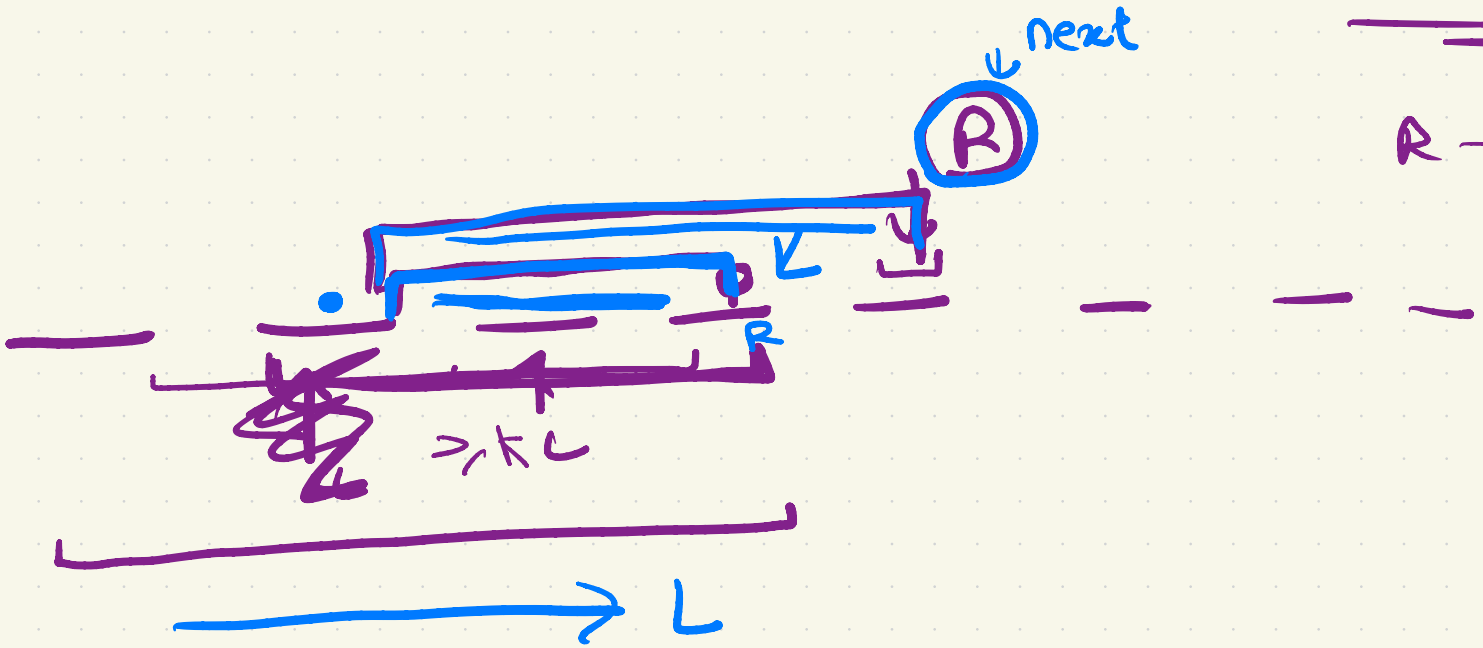
Problem

Given an array of N positive elements, find out the length of the shortest subarray with sum >= K.

Input: *arr[] = { 10, 5, 2, 7, 1, 9 }, k = 15*
Output: *2*
Explanation: *The sub-array is* ***{10, 5}****.*

SHORTEST Subarray sum ≥ k

$a_i > 0$



R - L + 1

Solution using 2 Pointers

If the current segment is not good any segment smaller will never be good, so add some elements to make it good.

If current segment is good, try removing an element and check if it still remains good.

Remove elements from left, add elements from right.

```
int n, k;
cin >> n >> k;
vector<int> a(n);
for(int i = 0; i < n; i++)
    cin >> a[i];
int ans = 1e9, sum = 0;
int left = 0, right = 0;
while(right < n){
    sum += a[right]; // include element on right
    while(left <= right && sum >= k){
        // update answer
        ans = min(ans, right - left + 1);
        // move left pointer 1 step right
        // remove a[left]
        sum -= a[left];
        left++;
    }
    right++; // move right pointer by 1 step right
}
cout << ans << endl;
```

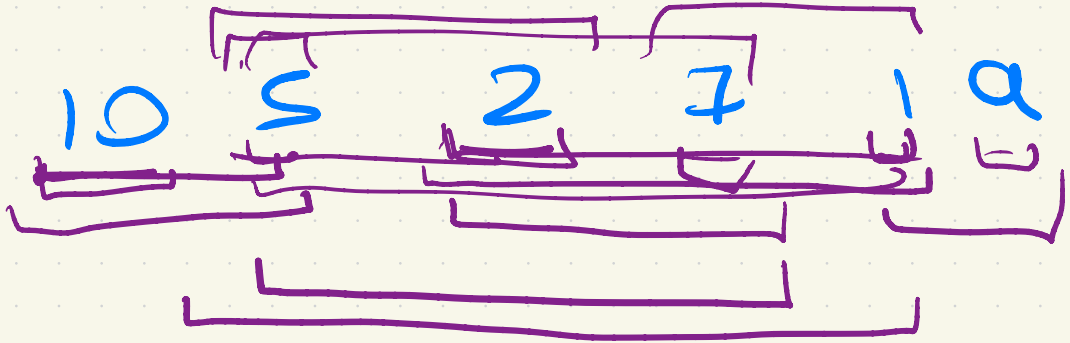
Good Segments Technique General Trick

- Condition 1: If Segment [L:R] is good then all the segments enclosed within in will be good
  - Use type 1 (Try to keep increasing segment size until it is good)
- Condition 2: If Segment [L:R] is good then all the segments enclosing it will be good
  - Use type 2 (Try to keep decreasing segment size until it is good)
- Do not use binary search for these problems now xD

Main adv. of Type 1 & Type 2

3.20

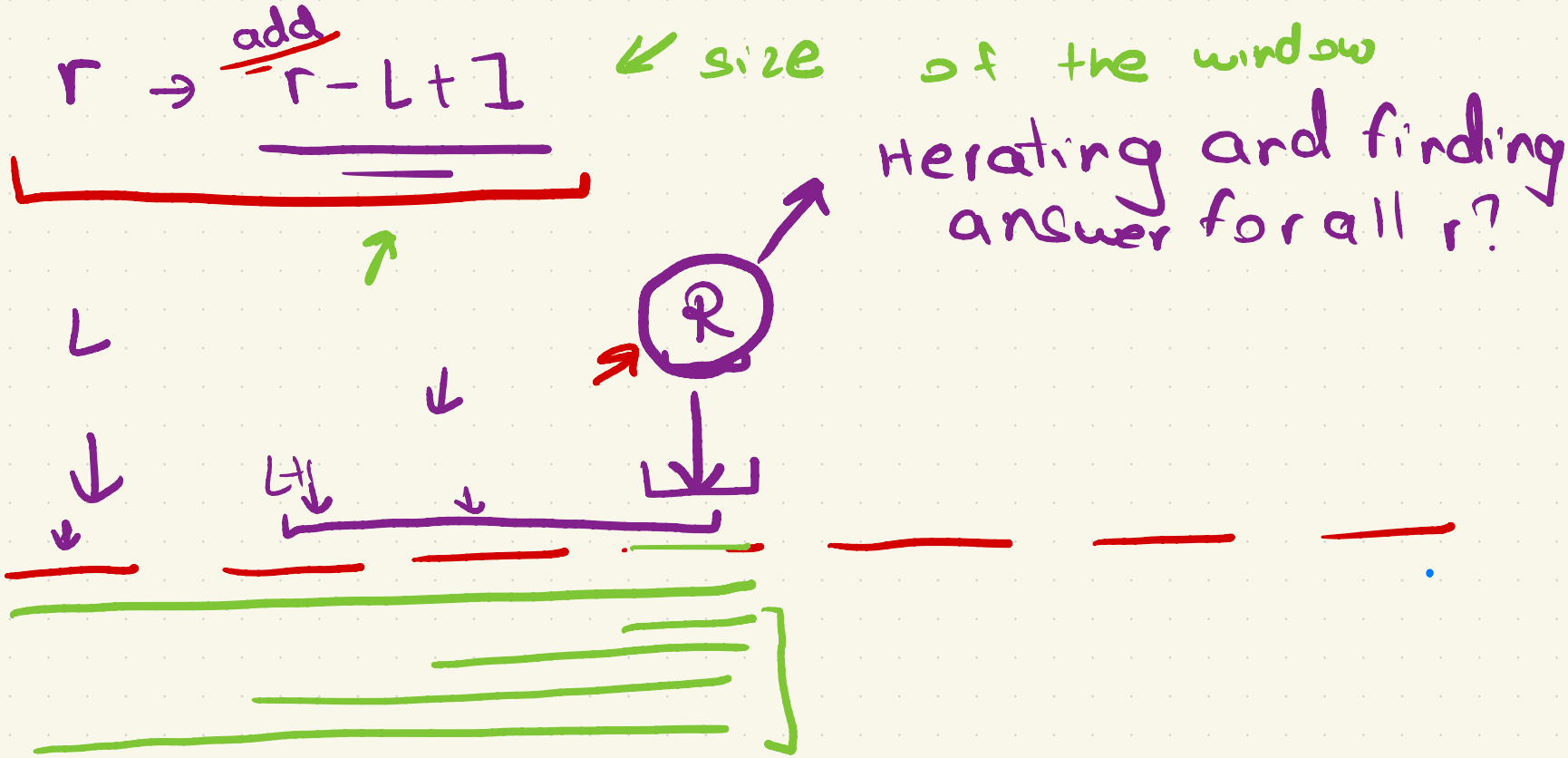
1. Find the number of subarrays with sum $\leq k$. $a_i > 0$

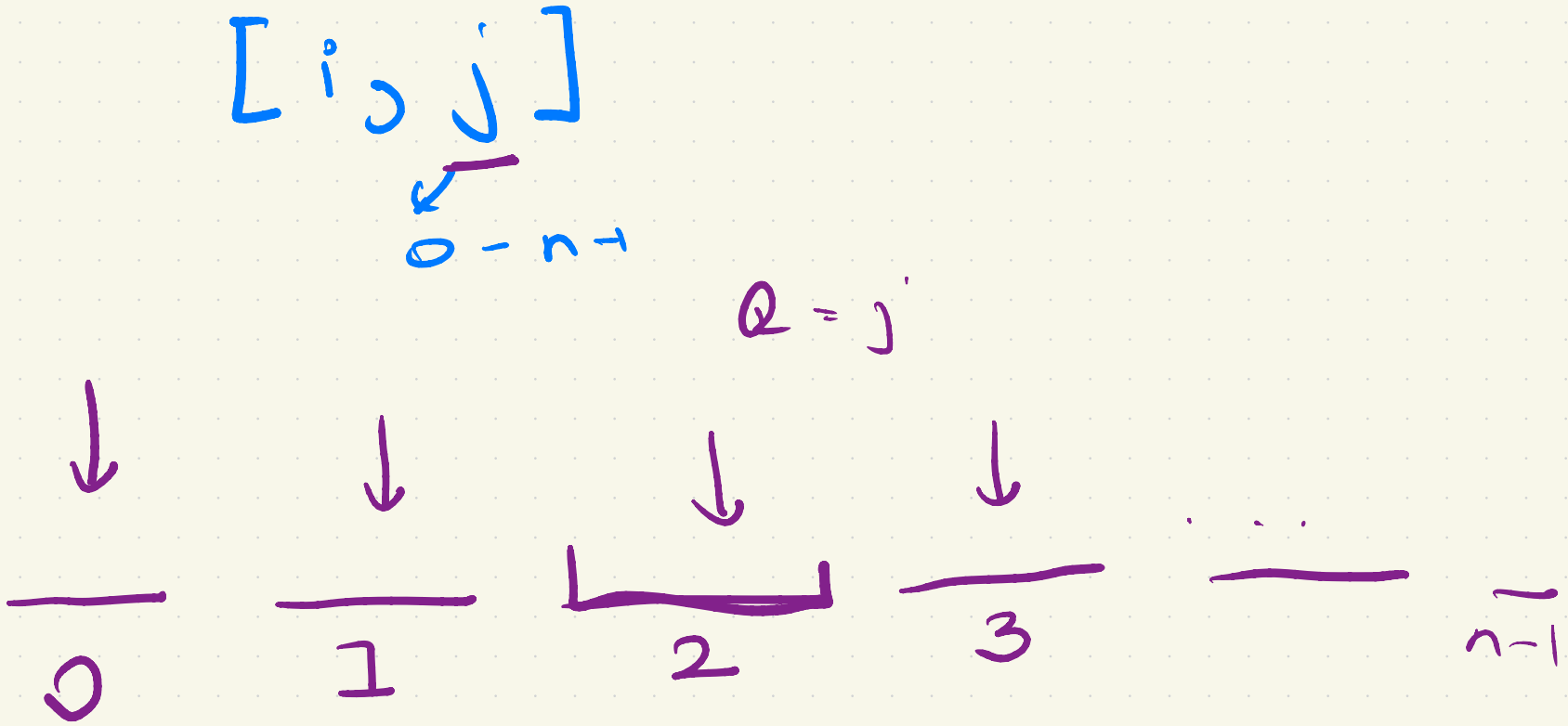


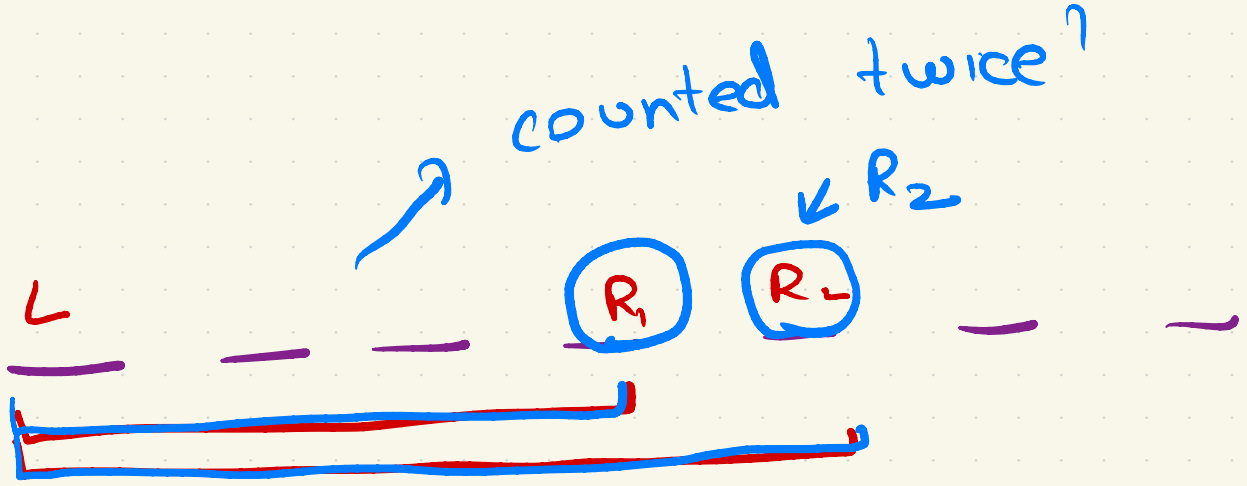
, $k = 15$

14

$6 + 5 + 2 + 1 = 14$



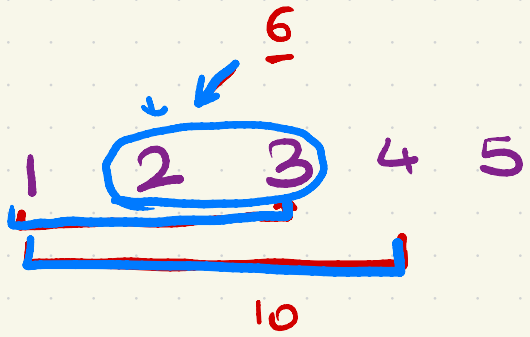




→ n * (n+1)/2 ⟶ r-l+1

why WA!

k = 10



2. Count number of subbarrays with sum $\geq k$, $a_i > 0$

Type 2

1 2 3 4 5

$k = 10$

4

1st solution ✓✓

student solution very nice!!!

Subarrays w/ sum ≥ k

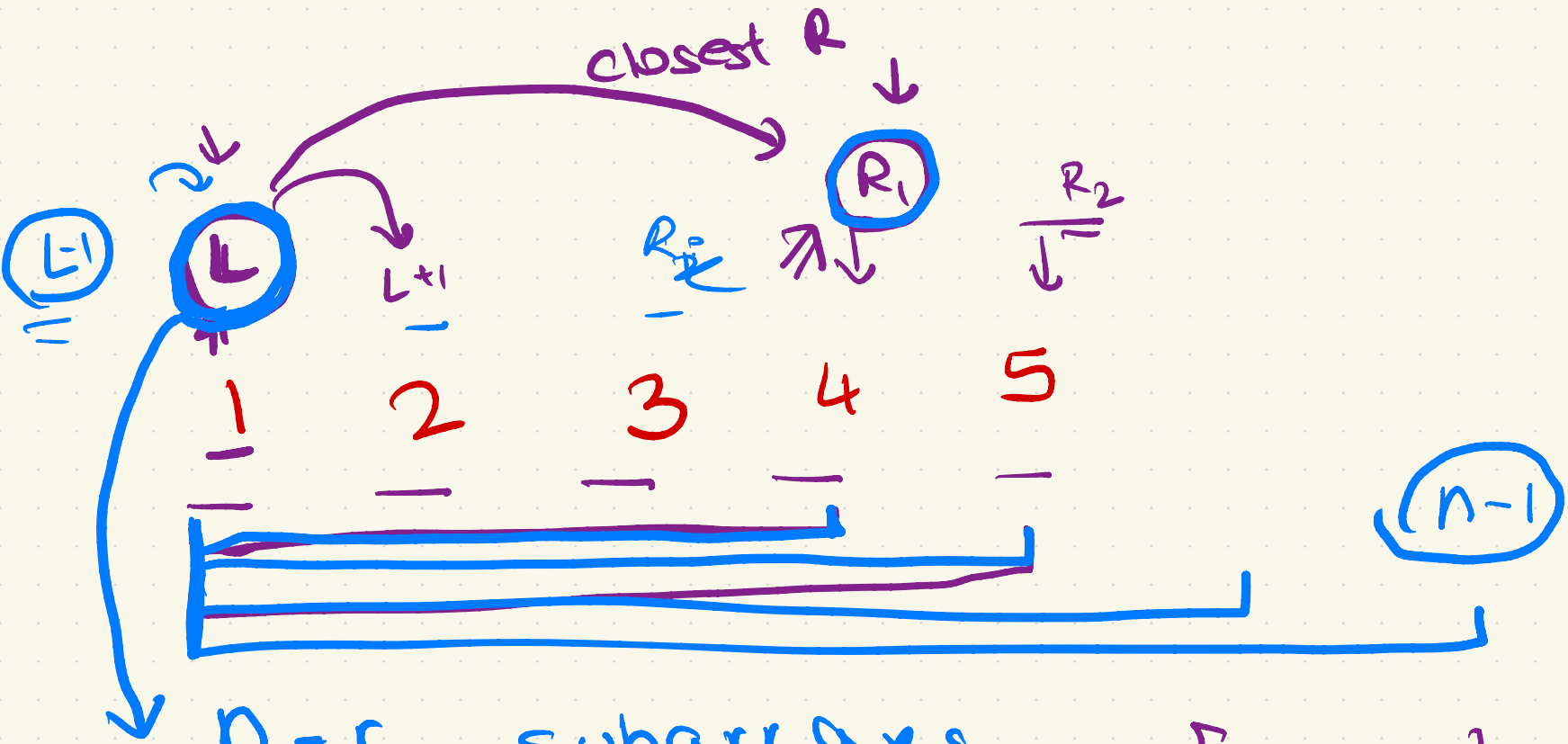
= Total - (subarrays w/ sum<k)

Previous Problem

↓

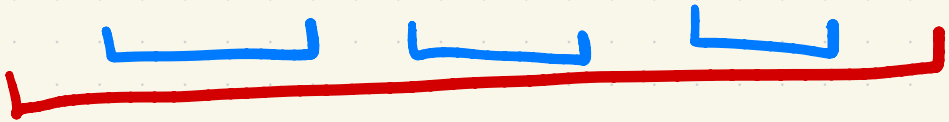
$\frac{n*(n+1)}{2}$ - count

Intended Solution. (Type 2)



$\frac{n-r}{(n-1-r+1)} \rightarrow n-r$ subarrays $[0, n-1]$

Type 1 :



Type 2 :

Number of Segments?

#Task
Code both of these problems

- How to find number of good segments?
- Let's solve the first problem - Number of subarrays with sum <= K
  - Simple! Just ~~multiply~~ add (j - i + 1) for every i
- Solve the last problem - Number of subarrays with sum >= K
  - Do it on your own [Homework]